

OUTPUTS:

WORKSHEET-6 (part-1)

```
# Run the test case
test_logistic_function()
```

All tests passed!

```
log loss(0, 0.1) ==> 0.10536051565782628
+++++-----+++++
log loss(1, 0.9) ==> 0.10536051565782628
```

```
# Run the test case
test_log_loss()
```

All tests passed!

```
# Run the test case
test_cost_function()
```

Test passed for simple case!

```
cost for logistic regression(X = [[ 10  20]
 [-10  10]], y = [1 0], w = [0.5 1.5], b = 1) = 5.500008350784907
```

```
Gradients computed successfully.
grad_w: [-4.99991649  4.99991649]
grad_b: 0.4999916492890759
```

```
Iteration 96600: Cost = 0.008550
Iteration 96700: Cost = 0.008541
Iteration 96800: Cost = 0.008532
Iteration 96900: Cost = 0.008523
Iteration 97000: Cost = 0.008514
Iteration 97100: Cost = 0.008505
Iteration 97200: Cost = 0.008496
Iteration 97300: Cost = 0.008487
Iteration 97400: Cost = 0.008478
Iteration 97500: Cost = 0.008469
Iteration 97600: Cost = 0.008460
Iteration 97700: Cost = 0.008452
Iteration 97800: Cost = 0.008443
Iteration 97900: Cost = 0.008434
Iteration 98000: Cost = 0.008425
Iteration 98100: Cost = 0.008416
Iteration 98200: Cost = 0.008408
Iteration 98300: Cost = 0.008399
Iteration 98400: Cost = 0.008390
Iteration 98500: Cost = 0.008382
Iteration 98600: Cost = 0.008373
Iteration 98700: Cost = 0.008364
Iteration 98800: Cost = 0.008356
Iteration 98900: Cost = 0.008347
Iteration 99000: Cost = 0.008339
Iteration 99100: Cost = 0.008330
Iteration 99200: Cost = 0.008321
Iteration 99300: Cost = 0.008313
Iteration 99400: Cost = 0.008304
Iteration 99500: Cost = 0.008296
Iteration 99600: Cost = 0.008287
Iteration 99700: Cost = 0.008279
Iteration 99800: Cost = 0.008270
Iteration 99900: Cost = 0.008262
Iteration 99999: Cost = 0.008254
```

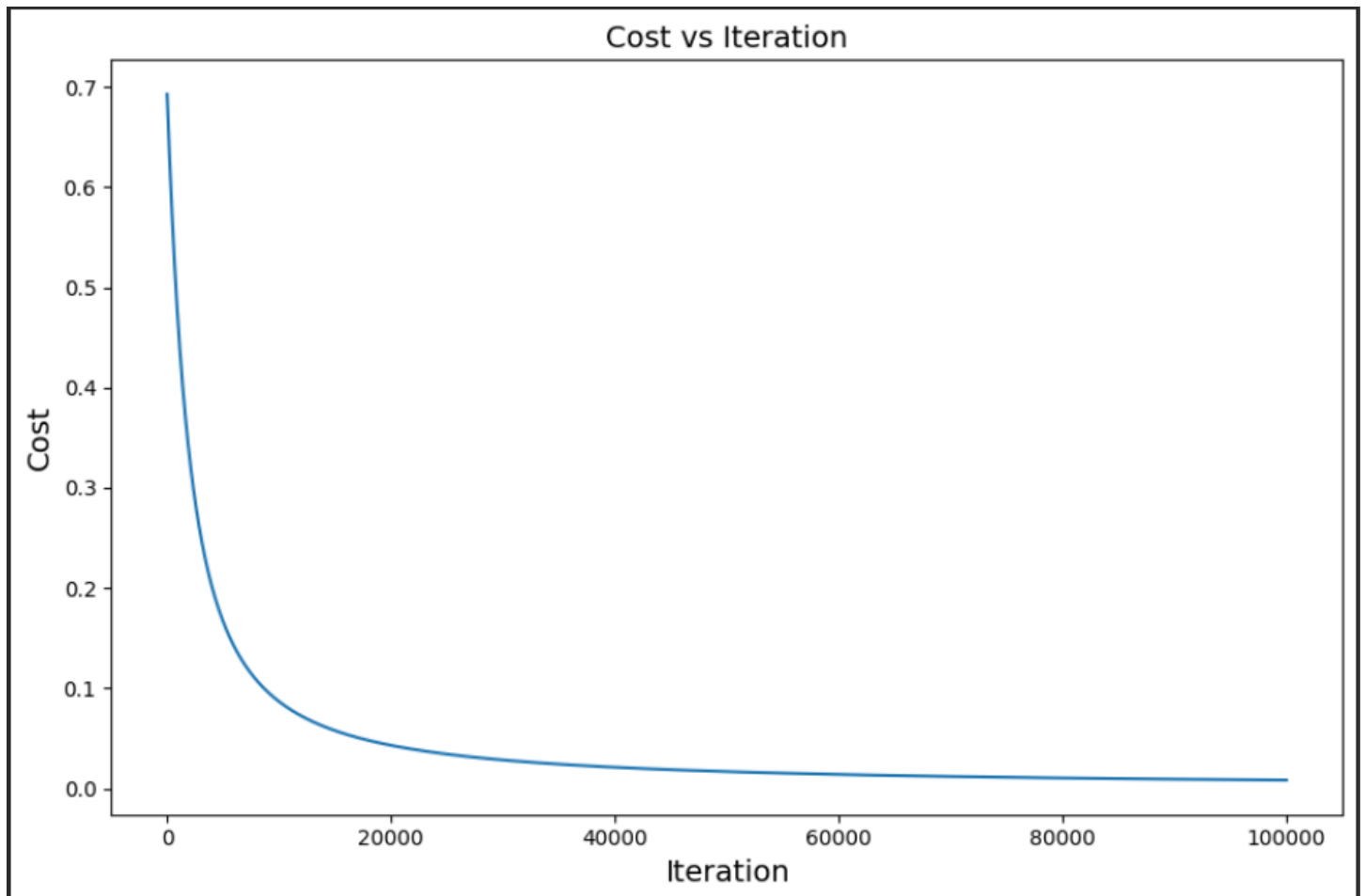
Final parameters:

w : [38.51304248 18.83386869], b : -2.8176836626325836

Iteration 0: w = [0.0005 0.00025], b = 0.000000

Iteration 99: w = [0.04992276 0.02489731], b = -0.000427

test_gradient_descent PASSED!



```
test_prediction()
```

Test passed!

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 768 entries, 0 to 767
```

```
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	Pregnancies	768 non-null	int64
1	Glucose	768 non-null	float64
2	BloodPressure	768 non-null	float64
3	SkinThickness	768 non-null	float64
4	Insulin	768 non-null	float64
5	BMI	768 non-null	float64
6	DiabetesPedigreeFunction	768 non-null	float64
7	Age	768 non-null	int64
8	Outcome	768 non-null	int64

```
dtypes: float64(6), int64(3)
```

```
memory usage: 54.1 KB
```

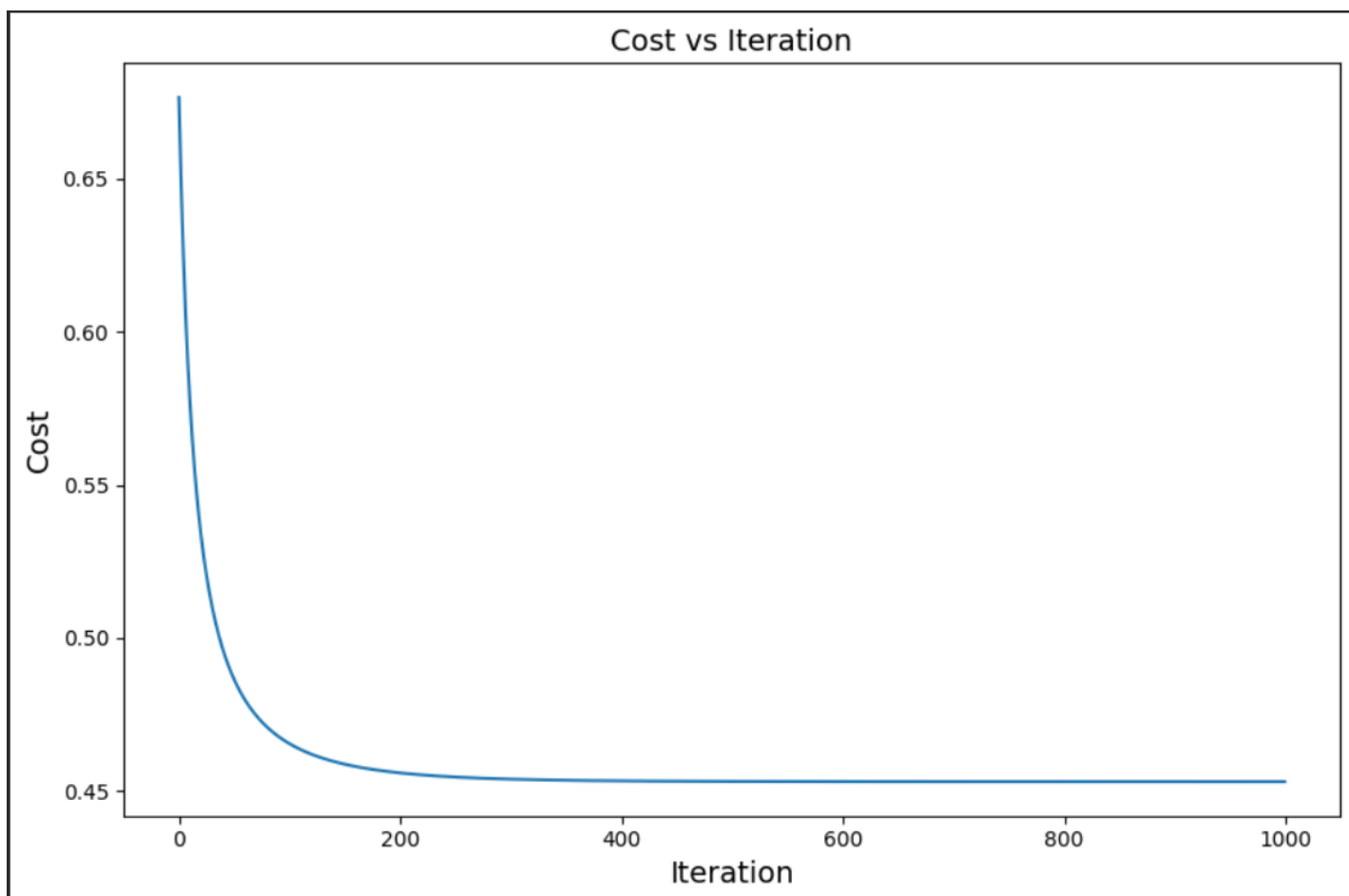
	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
count	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000	768.000000
mean	3.845052	121.656250	72.386719	29.108073	140.671875	32.455208	0.471876	33.240885	0.348958
std	3.369578	30.438286	12.096642	8.791221	86.383060	6.875177	0.331329	11.760232	0.476951
min	0.000000	44.000000	24.000000	7.000000	14.000000	18.200000	0.078000	21.000000	0.000000
25%	1.000000	99.750000	64.000000	25.000000	121.500000	27.500000	0.243750	24.000000	0.000000
50%	3.000000	117.000000	72.000000	29.000000	125.000000	32.300000	0.372500	29.000000	0.000000
75%	6.000000	140.250000	80.000000	32.000000	127.250000	36.600000	0.626250	41.000000	1.000000
max	17.000000	199.000000	122.000000	99.000000	846.000000	67.100000	2.420000	81.000000	1.000000

Training Logistic Regression Model:

```

Iteration 0: Cost = 0.676575
Iteration 100: Cost = 0.465441
Iteration 200: Cost = 0.455913
Iteration 300: Cost = 0.453874
Iteration 400: Cost = 0.453316
Iteration 500: Cost = 0.453148
Iteration 600: Cost = 0.453096
Iteration 700: Cost = 0.453079
Iteration 800: Cost = 0.453074
Iteration 900: Cost = 0.453072
Iteration 999: Cost = 0.453071

```



```

Train Loss (Cost): 0.4531
Test Loss (Cost): 0.5146

```

WORKSHEET-6 (part-2)

```
Test 1: All zeros
Passed.
Test 2: Simple case
Passed.
Test 3: Large identical values
Passed.
Test 4: Small identical values
Passed.
Test 5: Mixed positive and negative
Passed.
All tests passed for softmax function.
```

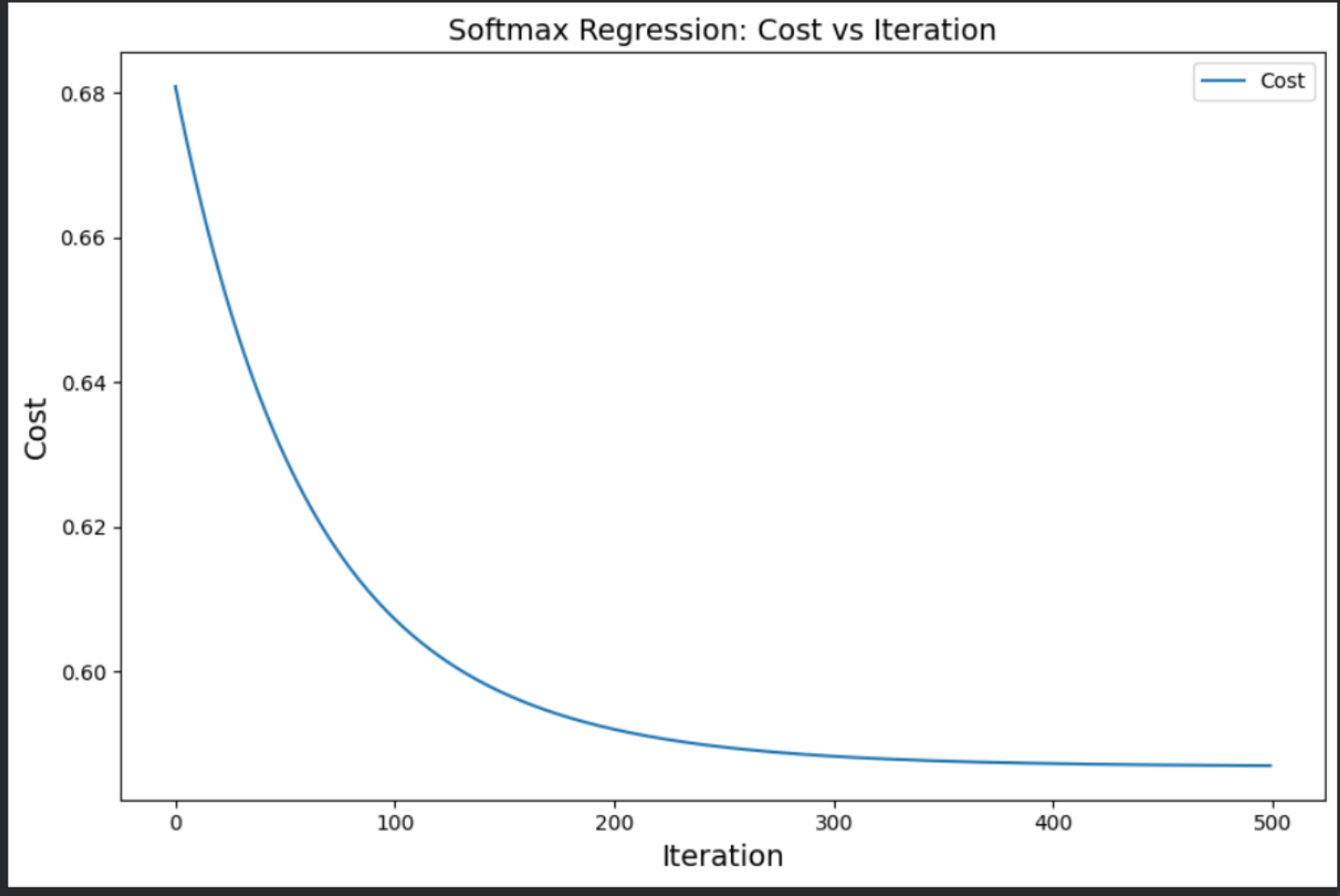
```
All test cases passed for loss_softmax!
```

```
All test cases passed for cost_softmax!
```

```
All tests passed for compute_gradient_softmax!
```

```
Iteration 0: Cost = 0.644539
Iteration 100: Cost = 0.637800
Iteration 200: Cost = 0.636906
Iteration 300: Cost = 0.636634
Iteration 400: Cost = 0.636551
Iteration 499: Cost = 0.636526
Optimized W: [[-0.10705903  0.10705903]
 [ 0.11305002 -0.11305002]]
Optimized b: [ 0.22010905 -0.22010905]
Final cost: 0.6365257252185855
```

Gradient descent softmax test PASSED!



Predicted labels: [1 1 1]
test_predict_softmax PASSED!

Confusion Matrix:
[[2 0]
[1 2]]
Precision: 1.0
Recall: 0.6666666666666666
F1-score: 0.8

Dataset Info:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 150 entries, 0 to 149
```

```
Data columns (total 4 columns):
```

#	Column	Non-Null Count	Dtype
0	sepal length (cm)	150 non-null	float64
1	sepal width (cm)	150 non-null	float64
2	petal length (cm)	150 non-null	float64
3	petal width (cm)	150 non-null	float64

```
dtypes: float64(4)
```

```
memory usage: 4.8 KB
```

```
None
```

Dataset Description Before Cleaning:

	sepal length (cm)	sepal width (cm)	petal length (cm)	\
count	150.000000	150.000000	150.000000	
mean	5.843333	3.057333	3.758000	
std	0.828066	0.435866	1.765298	
min	4.300000	2.000000	1.000000	
25%	5.100000	2.800000	1.600000	
50%	5.800000	3.000000	4.350000	
75%	6.400000	3.300000	5.100000	
max	7.900000	4.400000	6.900000	

	petal width (cm)
count	150.000000
mean	1.199333
std	0.762238
min	0.100000
25%	0.300000
50%	1.300000
75%	1.800000
max	2.500000

Shape assertions passed!

Training Softmax Regression Model:

```
Iteration 0: Cost = 1.006823
```

```
Iteration 100: Cost = 0.319428
```

```
Iteration 200: Cost = 0.251376
```

```
Iteration 300: Cost = 0.211741
```

```
Iteration 400: Cost = 0.185004
```

```
Iteration 500: Cost = 0.165744
```

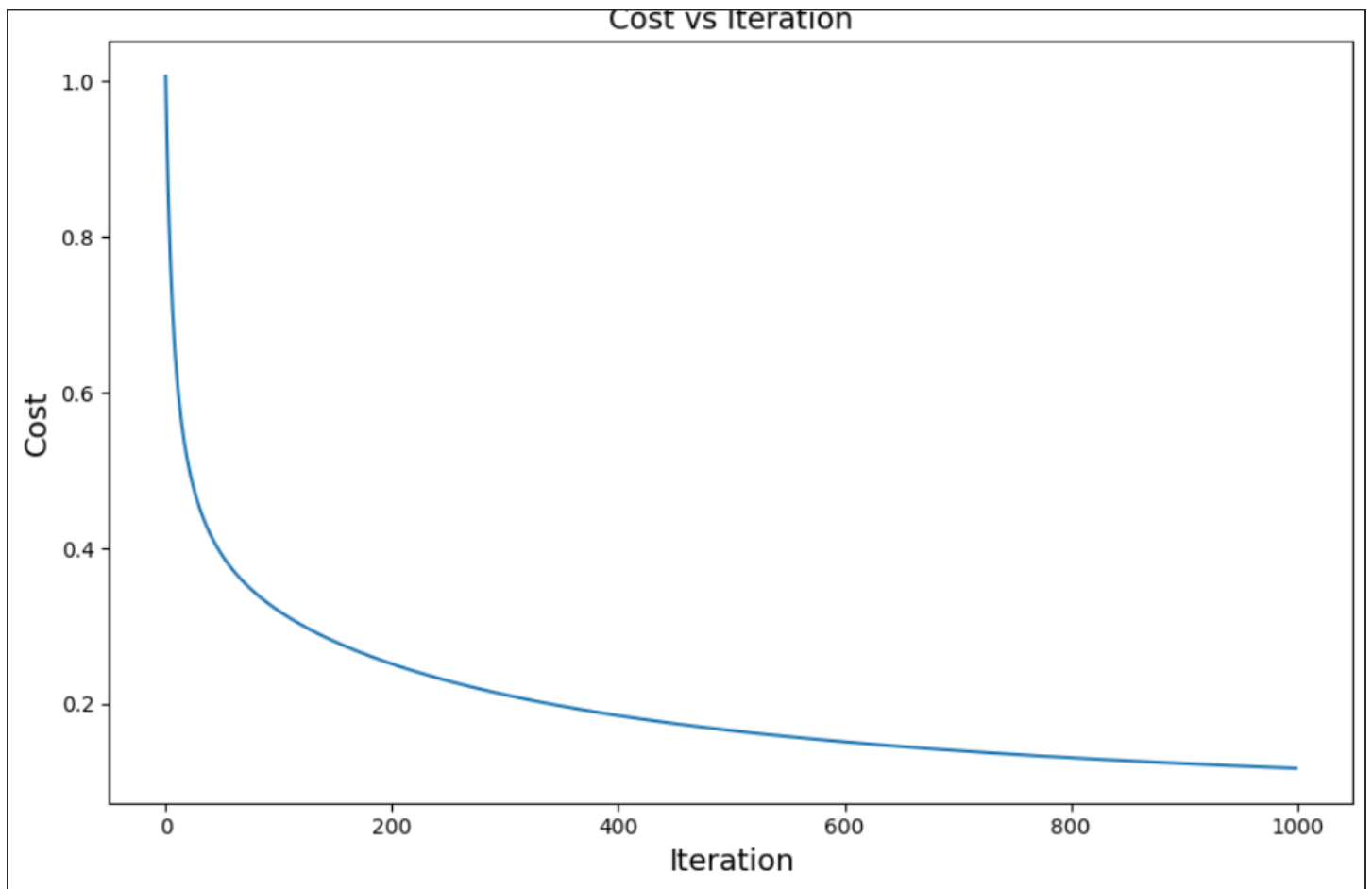
```
Iteration 600: Cost = 0.151226
```

```
Iteration 700: Cost = 0.139898
```

```
Iteration 800: Cost = 0.130812
```

```
Iteration 900: Cost = 0.123360
```

```
Iteration 999: Cost = 0.117192
```



```
Train Loss (Cost): 0.1172
Test Loss (Cost): 0.1575
```

```
Test Accuracy: 93.33%
```

```
Confusion Matrix:
```

```
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]
```

```
Precision: 0.93
```

```
Recall: 0.93
```

```
F1-Score: 0.93
```

Confusion Matrix

